Salahaddin University-Erbil
College of Computer Science and IT
Stage 2 Cs & IT

Web Technology Basics - Lec. 2 – CSS

Mrs. Saja Ataallah Muhammed
Email: saja.muhammed@su.edu.krd
BSc in Computer Science/ Baghdad University
MSc in Computer Science/ Salahaddin University

Tuesday: IT- 8:30-10:30/ Hall 3
CS- 10:30 -12:30/ Hall 16

What you will gain from this course:

- Style and layout pages with CSS.
- Make pages responsive, accessible, and performant.

CSS Styles – Colors & Backgrounds

1. **Color:** sets the color of the text.
   Color: either color name or rgb(**0-255**, **0-255**, **0-255**) or #hex code **#FF0000** or **#F00**;
2. **Background-color:** sets the background color
   Background-color: either color name or rgb(0-255, 0-255, 0-255) or #hex code;
3. **Background-image:** sets an image as background.
   Background-image: URL(ImageName.Image Ext.);
4. **Background-repeat:** repeats the background image.
   Background-repeat: repeat-x; **or** repeat-y; **or** repeat(default); **or** round; **or** no-repeat;

*Round value The background-image is repeated and squished or stretched to fill the space (no gaps).

CSS Styles – Colors & Backgrounds

5. **Background-position:** sets the position of background image.

   Background-position: horizontal –offset vertical-offset;

6. **Background-attachment:** a property that determines whether the background image should be scrolled with page content or fixed.

   Background-attachment:either scroll; **or** fixed;

7. **Background-size:** determines the size of background image.

   Background-size: width height; **or** contain; **or** cover;

- **Contain** Scale the image so that both its width and height fit into the background positioning area.
- **Cover** Scale the image so that both its width and height can completely cover the background positioning area.

CSS Styles – Colors & Backgrounds

8. **Background-origin:** determines whether to start the background image from the upper left corner of the border or from the upper left corner of the padding edge or from upper left corner of the content.

Background-origin:either content-box; **or** border-box **or** padding-box(default);

9. **Multiple Backgrounds**

Background: URL(Image1 Name. Ext.) ,
URL(Image2 Name. Ext.),
URL(Image3 Name. Ext.), …..,
URL(Image N Name.Ext.);

CSS Styles – Floating & Fixed Position

1. **Float:** property specifies how an element should float.

   Float: either left; **or** right; **or** none(default);

- **Note:** Absolutely positioned elements ignore the float property!
- **Note:** Elements after a floating element will flow around it. To avoid this, use the clear property.

2. **Clear:** The clear property specifies on which sides of an element floating elements are not allowed to float.

   Clear: either left; **or** right **or** none(default);

3. **Position:** property specifies the type of positioning method used for an element.

CSS Styles – Floating & Fixed Position

Position: either static(default); **or** fixed; **or** absolute; **or** sticky; Value of specific position could be set through:

Top: length in px, em inch or percentage;

Right: length in px, em inch or percentage;

Bottom: length in px, em inch or percentage;

Left: length in px, em inch or percentage;

4.Overflow property specifies what should happen if content overflows an element's box. This property specifies whether to clip content or to add scrollbars when an element's content is too big to fit in a specified area.

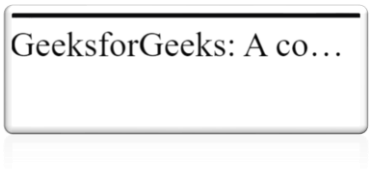
Note: The overflow property only works for block elements with a specified height.

Overflow: either visible(default); **or** hidden; **or** scroll; **or** auto;

text-overflow: property in CSS is used to specify that some text has overflown and hidden from view.

Example:

```
<html>
<head>  <style>
   div {
      width: 500px;
      font-size: 50px;
      white-space: nowrap;
      overflow: hidden;
      text-overflow: ellipsis;
   }
   </style>
</head>
<body>
   <div> GeeksforGeeks: A computer science portal for geeks. </div>
</body> </html>
```



Visibility & Stack Order

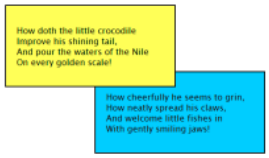
5. **Visibility** property specifies whether or not an element is visible.

   Visibility: either visible(visible);**or** hidden; **or** collapse;

6. **Z-index** property specifies the stack order of an element. An element with greater stack order is always in front of an element with a lower stack order.

➢**Note:** z-index only works on positioned elements (position:absolute, or position:fixed).

Z-index: integer number; **or** auto(default);



CSS Styles – Borders

1. **Border property** is a shorthand property for: border-width, border-style (required), and border-color
2. **Border-width** property sets the width of an element's four borders. This property can have from one to four values.

Synta:**border-width:** medium|thin|thick|length|inherit;

Examples:

➢**border-width: thin medium thick 10px;**

– top border is thin, right border is medium, bottom border is thick, left border is 10px

➢**border-width: thin medium thick;**

– top border is thin, right and left borders are medium, bottom border is thick

CSS Styles – Borders

➢**border-width: thin medium;**

– top and bottom borders are thin, right and left borders are medium

➢**border-width: thin;**

– all four borders are thin

Note: Always declare the border-style property before the border-width property. An element must have borders before you can set the width.

3. **Border-style** property sets the style of an element's four borders. This property can have from one to four values.

Syntax: **border-style: (required)**

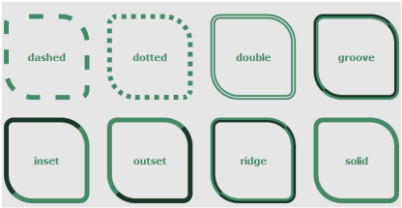
none|hidden|dotted|dashed|solid|double|groove|ridge|inset|outset|inherit;

Note: 3D borders like inset, riged groove and outset effects depends on the border color value

CSS Styles – Borders

Examples:

- **border-style: dotted solid double dashed;**
  - top border is dotted, right border is solid, bottom border is double, left border is dashed
- **border-style: dotted solid double;**
  - top border is dotted, right and left borders are solid, bottom border is double
- **border-style: dotted solid;**
  - top and bottom borders are dotted,
  - right and left borders are solid
- **border-style: dotted;**
  - all four borders are dotted



CSS Styles – Borders

3. **Border-color** property sets the color of an element's four borders. This property can have from one to four values.

Syntax: border-color: color|transparent|inherit;

Note: If border-color is omitted, the color applied will be the color of the text.

- **If the border-color property has four values:**

  border-color: red green blue pink;

– top border is red, right border is green, bottom border is blue, left border is pink

- **If the border-color property has three values:**

  border-color: red green blue;

– top border is red, right and left borders are green, bottom border is blue

- **If the border-color property has two values:**

  border-color: red green;

– top and bottom borders are red, right and left borders are green

CSS Styles – Borders

➢**If the border-color property has one value:**

border-color: red;

– all four borders are red

Note: Always declare the border-style property before the border-color property. An element must have borders before you can change the color.

4. Border-bottom property is a shorthand property for (in the following order):

➢**border-bottom-width, border-bottom-style, border-bottom-color**

```
h1 {
  border-bottom: 5px solid red;
}
h2 {
  border-bottom: 4px dotted blue;
}
```

CSS Styles – Borders

5. Border-left property is a shorthand property for (in the following order):

➢**border-left-width, border-left-style, border-left-color**

6. Border-right: property is a shorthand property for (in the following order):

border-right-width, border-right-style (required), border-right-color(same as left & bottom)

7. Border-top: property sets all the top border properties in one declaration.

The properties that can be set must be in the following order:

border-top-width, border-top-style (required), border-top-color(same as bottm & left

Ex:

```
h1 {
  border-right: 5px solid red;
}
h2 {
  border-left: 4px dotted blue;
}
```

CSS Styles – Borders

8. **Border-radius:** property defines the radius of the element's corners.

Note: This property allows you to add rounded corners to elements!

This property can have from one to four values. Here are the rules:

- **Four values - border-radius**: 15px 50px 30px 5px;

(first value applies to top-left corner, second value applies to top-right corner, third value applies to bottom-right corner, and fourth value applies to bottom left corner)

- **Three values - border-radius**: 15px 50px 30px;

(first value applies to top-left corner, second value applies to top-right and bottom-left corners, and third value applies to bottom-right corner):

- **Two values - border-radius:** 15px 50px;

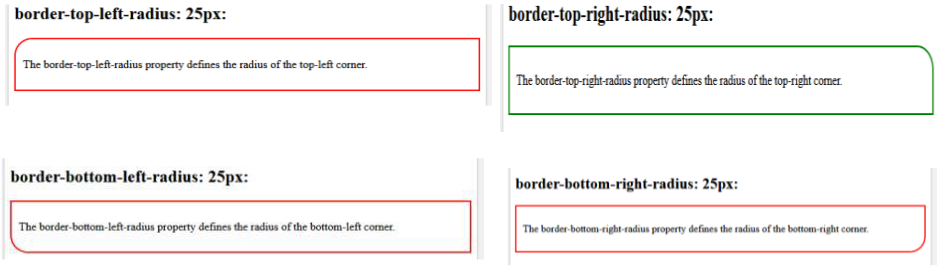
(first value applies to top-left and bottom-right corners, and the second value applies to top-right and bottom-left corners)

- **One value - border-radius:** 15px;

(the value applies to all four corners, which are rounded equally.

CSS Styles – Borders

9. **Border-top-left-radius**: length | inherit;
10. **Border-top-right-radius:** length | inherit;
11. **Border-bottom-left-radius:** length | inherit;
12. **border-bottom-right-radius**: length | inherit;



CSS Styles – Transform

- A simple example of a 2D transform is scaling down and rotating an image in its normal state. When this happens with just a transform, the change is instantaneous.
- CSS transitions can be applied to a wide range of properties, and are supported by all mainstream browsers except IE 9 and earlier.

Function	Description
translateX()	Moves the element along the X axis by the specified length or percentage.
translateY()	Moves the element along the Y axis by the specified length or percentage.
translate()	Moves the element from its default position. Takes two comma-separated lengths or percentages. The first value represents the horizontal offset and the second the vertical offset. If only one value is given, the element is moved only horizontally.
rotate()	Rotates the element around its anchor point by the specified angle.
scaleX()	Scales the element along the X axis by the specified scaling factor.
scaleY()	Scales the element along the Y axis by the specified scaling factor.
scale()	Scales the element. Takes one or two numbers as the scaling factor. If one number is given, both axes are scaled by the same multiplier. If two comma-separated numbers are given, the first applies to the X axis and the second to the Y axis.
skewX()	Skews the element along the X axis by the specified angle.
skewY()	Skews the element along the Y axis by the specified angle.

CSS Styles – Boxes

1. **Margin**: length | inherit;

Property	Initial Value	Inherited	Description
margin-top	0	No	Sets the top margin of an element.
margin-right	0	No	Sets the right margin of an element.
margin-bottom	0	No	Sets the bottom margin of an element.
margin-left	0	No	Sets the left margin of an element.
margin	0	No	Shorthand property. Accepts between one and four values.

2. **Padding**

Property	Initial Value	Inherited	Description
padding-top	0	No	Adds padding to the top of an element.
padding-right	0	No	Adds padding to the right of an element.
padding-bottom	0	No	Adds padding to the bottom of an element.
padding-left	0	No	Adds padding to the left of an element.
padding	0	No	Shorthand property. Accepts between one and four values.

CSS Styles – Boxes

3. **Box-shadow**: The box-shadow property adds one or more drop shadows to an element. You specify a shadow using 2–4 space separated lengths, which represent the following values (in this order):

Horizontal offset: A positive value positions the shadow to the right. A negative value positions it to the left.

Vertical offset: A positive value moves the shadow down. A negative value moves it up.

Blur radius: If omitted or set to 0, the shadow has a crisp edge. Negative values are not allowed.

Spread distance: A **positive value** causes the shape to expand in all directions. While a **negative value** (used with an inset shadow) causes it to contract.

- In addition to these lengths, you can specify a **color.** Otherwise, the shadow is the same color as the element's text.
- By default, the shadow is drawn outside the element, but you can draw it inside the element by adding the **inset** keyword.
- To specify multiple shadows on the same element, separate the definition of each shadow with a comma.

box-shadow: none| h-offset v-offset blur spread color |insetl|inherit;

CSS Styles – Boxes

- Drop Shadow

box-shadow: 5px 5px 7px red;

Dropped shadow.

- Reverse Shadow

box-shadow: -5px -5px 7px red;

Reversed box-shadow

- Hidden Shadow

-box-shadow: 5px 5px 7px 4px grey;

Hidden Spread

- Visible Shadow

box-shadow: 5px 5px 7px 12px grey;

Visible Spread

CSS Styles – Boxes

Examples:

➢ **Multiple Shadows**

box-shadow: 5px 5px blue, 10px 10px red, 15px 15px green:

➢ **Positive inset shadow**

box-shadow: inset 10px 10px 10px grey;

➢ **Negative inset shadow**

box-shadow: inset -10px -10px 10px grey;

Note The most recent versions of most browsers support the standard box-shadow property. However, Android4 (Ice Cream Sandwich), Safari 5, and older versions of iOS Safari need the -webkit browser-specific prefix.

CSS Styles – Boxes

4. **Box-decoration-break:** this property specifies how the background, padding, border, border-image, box-shadow, margin, and clip-path of an element is applied when the box for the element is fragmented.

box-decoration-break: slice(default)| clone| inherit;

✓ **slice :**Default. Box decorations are applied to the element as a whole and break at the edges of the element fragments.

✓ **Clone:** Box decorations apply to each fragment of the element as if the fragments were individual elements. Borders wrap the four edges of each fragment of the element, and backgrounds are redrawn in full for each fragment.

In HTML File

CSS
is
easy
to learn

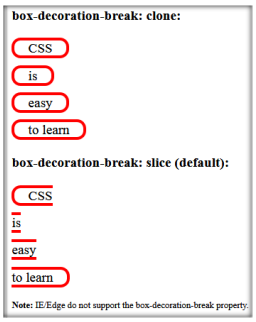
In CSS File:

span { border: 5px solid red; padding: 0px 16px;

border-radius: 16px; font-size: 24px; line-height: 2;

box-decoration-break: **clone**; } or

//box-decoration-break: **slice;**



CSS Styles – Tables

Property	Initial Value	Inherited	Description
border-collapse	separate	Yes	By default, the table and each cell have independent borders. Borders are merged by setting the value of this property to collapse.
border-spacing	0	Yes	Controls the spacing between table cells. Horizontal and vertical spacing can be controlled independently.
caption-side	top	Yes	Determines whether the table caption is displayed above or below the table.
empty-cells	show	Yes	If set to hide, turns off the display of borders and backgrounds associated with empty table cells.
table-layout	auto	No	If set to fixed, the width of each column is set in the column definitions or first row.
text-align	left	Yes	Controls the horizontal alignment of content in table cells. With right-to-left languages, the default is right.
vertical-align	middle	No	Controls vertical alignment in table cells.

- **border-collapse: separate;** for separate border **or** **collapse ;** for merged borders
- **border-spacing:** H-space V-space;
- **caption-side:** top or bottom;
- **empty-cells:**show **or** hide; to hide all empty cells with their borders and backgrounds

CSS Styles – Tables

- **nth-child(even or odd):** adds styles to all even (or odd) table rows

Ex: tr:nth-child(even) {background-color: gray;}

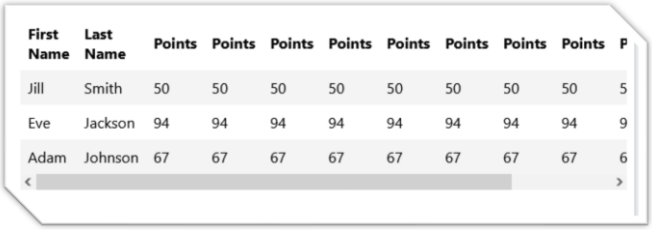
- **overflow-x** :A responsive table will display a horizontal scroll bar if the screen is too small to display the full content: Add a container element (like) with overflow-x:auto around the <table> element to make it responsive:

Example:

```
<head>
<style>
tr : nth-child(even){
background-color: #f2f2f2;“}
</style>
</head>
<body>

<table>
... table content ...
</table>

</body>
```



First Name	Last Name	Points	Points	Points	Points	Points	Points	Points	Points	P
Jill	Smith	50	50	50	50	50	50	50	50	5
Eve	Jackson	94	94	94	94	94	94	94	94	9
Adam	Johnson	67	67	67	67	67	67	67	67	6

CSS Styles – Lists

Property	Initial Value	Description
list-style-type	disc	Determines the type of symbol used. By default, unordered lists use a solid disc, and ordered lists use a number followed by a period (dot).
list-style-position	outside	Controls whether the symbol or number is displayed as a hanging indent (default) or nested inside the list item.
list-style-image	none	Allows you to use your own image in place of an automatically generated symbol.
list-style	See individual properties	Shorthand property that lets you specify all properties in a single declaration.

- **list-style-type:** disc| circle| sqaure | decimal |lower-alpha | upper-alpha | lower-latin |upper-latin;……,etc.
- **list-position:** inside;or outside;
- **list-style-image**: url(imgName.type);
- **list-style:** square inside url("sqpurple.gif");

- Coffee - A brewed drink prepared from roasted coffee beans...
- Tea
- Coca-cola

- Coffee - A brewed drink prepared from roasted coffee beans...
- Tea
- Coca-cola

CSS Styles – Images

- **Opacity:** The opacity property can take a value from 0.0 - 1.0. The lower value, the more transparent.
- **Note:** IE8 and earlier use **filter:alpha(opacity=x)**. The x can take a value from 0 - 100. A lower value makes the element more transparent.

opacity 0.2

opacity 0.5

opacity 1

- **Filter:** The CSS filter property adds visual effects (like blur and saturation) to an element.

Ex: filter: contrast(100%);

filter: grayscale(100%);

Note: The filter property is not supported in Internet Explorer, Edge 12, or Safari 5.1 and earlier.

CSS Styles – Border Images

- With the CSS **border-image** property, you can set an image to be used as the border around an element.
- The property has three parts:

1.The image to use as the border

2.Where to slice the image

3.Define whether the middle sections should be repeated or stretched

- We will use the following image (called "border.png"):

- The border-image property takes the image and slices it into nine sections, like a tic-tac-toe board. It then places the corners at the corners, and the middle sections are repeated or stretched as you specify.

Note: For border-image to work, the element also needs the border property set!

Note: Internet Explorer 10, and earlier versions, do not support the border-image property.

Tip: The border-image property is actually a shorthand property for the border-image-source, border-image-slice, border-image-width, and border-image-repeat properties.

CSS Styles – Border Images

- The **border-image-source** property specifies the path to the image to be used as a border (instead of the normal border around an element).

  border-image-source: url(border.png);

- The **border-image-slice** property specifies how to slice the image specified by border-image-source. The image is always sliced into nine sections: four corners, four edges and the middle.

- The "middle" part is treated as fully transparent.

  border-image-slice: 30%;

- The **border-image-repeat** property specifies whether the border image should be repeated, rounded or stretched.

- **Round value:** The image is tiled (repeated) to fill the area. If it does not fill the area with a whole number of tiles, the image is rescaled so it fits

- The **border-image-width** property specifies the width of the border image.

  border-image-width: 10px;

CSS Styles – Border Images

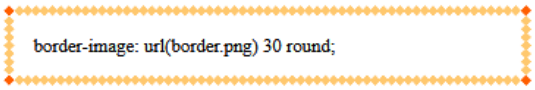
In CSS File:

```
#borderimg {
border: 10px solid transparent;
padding: 15px;
border-image: url(border.png) 30 round;
```

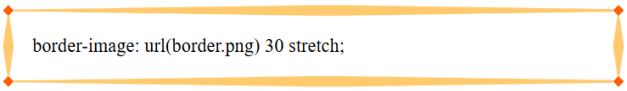
HTML File:

```
<p>Here, the middle sections of the
    image are repeated to create the
    border:</p>
<p id="borderimg">border-image:
  url(border.png) 30 round;</p>
<p>Here is the original image:</p>
<img src="border.png">
```

Here, the middle sections of the image are repeated to create the border:



Here is the original image:



CSS Styles – Transform

- A simple example of a 2D transform is scaling down and rotating an image in its normal state. When this happens with just a transform, the change is instantaneous.
- CSS transitions can be applied to a wide range of properties, and are supported by all mainstream browsers except IE 9 and earlier.

Function	Description
`translateX()`	Moves the element along the X axis by the specified length or percentage.
`translateY()`	Moves the element along the Y axis by the specified length or percentage.
`translate()`	Moves the element from its default position. Takes two comma-separated lengths or percentages. The first value represents the horizontal offset and the second the vertical offset. If only one value is given, the element is moved only horizontally.
`rotate()`	Rotates the element around its anchor point by the specified angle.
`scaleX()`	Scales the element along the X axis by the specified scaling factor.
`scaleY()`	Scales the element along the Y axis by the specified scaling factor.
`scale()`	Scales the element. Takes one or two numbers as the scaling factor. If one number is given, both axes are scaled by the same multiplier. If two comma-separated numbers are given, the first applies to the X axis and the second to the Y axis.
`skewX()`	Skews the element along the X axis by the specified angle.
`skewY()`	Skews the element along the Y axis by the specified angle.

CSS Styles – Transform

➢ **Transform:** none | transform-functions | inherit;

Examples:

Transform: rotate(90deg); *2D

Transform: skewY(20deg);

Transform: scaleY(1.5);

Transform: translateX(10px);

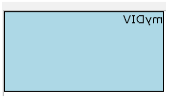
Transform: scale(2,3);

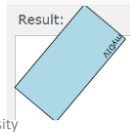
Transform: translate(10px,20px);

Transform: skew(20deg,30deg);

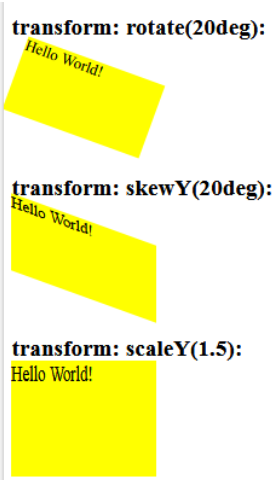
➢ 3D rotation

Transform: rotateY(180deg); **Transform:** rotateX(180deg);





- **Transform:** rotateZ(130deg);



CSS Styles – Transform

➢ The **transform-origin** property allows you to change the position of transformed elements.

➢ 2D transformations can change the x- and y-axis of an element. 3D transformations can also change the z-axis of an element.

➢ By default, 2D transforms originate from the centre of an element. With translate(), translateX(), and translateY(), it makes no difference where the movement is calculated from. The result is always the same.

➢ However, with the other transform functions, it can make a significant difference.

Syntax:

Transform-origin: x-axis y-axis z-axis | inherit;

x-axis left, center, right, length, %

y-axis top, center, bottom, length, %

z-axis length

CSS Styles – Transition

- Is animating a smooth change between one state and another over a specified duration.
- For example, you can scale and rotate the image over a period of one second. When you move the mouse away from the image, the transition is reversed, returning to its original state over the same duration.
- Transitions can be applied to all elements
- The transition property is a shorthand property for:
  - transition-property
  - transition-duration
  - transition-timing-function
  - transition-delay

CSS Syntax

transition: property duration timing-function delay | inherit;

Note: Always specify the transition-duration property, otherwise the duration is 0s, and the transition will have no effect.

CSS Styles – Transition

- The **transition-property** specifies the name of the CSS property the transition effect

- Transition-property: none | all |property | inherit;

- The **transition-duration** property specifies how many seconds (s) or milliseconds (ms) a transition effect takes to complete.

-**Transition-duration**: time | inherit;

- The **transition-timing-function** property specifies the speed curve of the transition effect.

-**Transition-timing-function:** linear|ease|ease-in|ease-out|ease-in-out|step-start|step-end|steps(int,start|end)|inherit

CSS Styles – Transition

- linear specifies a transition effect with the same speed from start to end.
- Ease specifies a transition effect with a slow start, then fast, then end slowly (this is default)
- ease-in Starts slow and speeds up.
- ease-out Starts fast and slows toward the end.
- ease-in-out Similar to ease, but slower at each end.
- steps(int,start|end) Specifies a stepping function, with two parameters. The first parameter specifies the number of intervals in the function. It must be a positive integer (greater than 0). The second parameter, which is optional, is either the value "start" or "end", and specifies the point at which the change of values occur within the interval. If the second parameter is omitted, it is given the value "end“.

Transition-timing-function: Steps(2,end) ; 2 intervals, start at end

CSS Styles – Transition

- The **transition-delay** property specifies when the transition effect will start.
- The transition-delay value is defined in seconds (s) or milliseconds (ms).

  Transition-delay: time | inherit;

```
div {
width: 100px;
height: 100px;
background: red;
transition: width 2s;
}
div:hover {
width: 300px;
}
```

CSS Styles – Animatable

- Some CSS properties are animatable, meaning that they can be used in animations and transitions. These properties can change gradually from one value to another, like size, numbers, percentage and color.
- The CSS Animations allow you to define keyframes that specify what an element's properties should be at each stage of the animation.
- You can specify how many times the animation should run or make it repeat endlessly.
- In spite of their power, CSS animations aren't designed for heavy graphic manipulation.

CSS Styles – Animatable

- The **animation** property is a shorthand property for:
  - animation-name
  - animation-duration
  - animation-timing-function
  - animation-delay
  - animation-iteration-count
- **Syntax**

animation: myfirst 5s linear 2s infinite alternate;

- ✓ **animation-name :** Specifies the name of the keyframe you want to bind to the selector
- ✓ **animation-duration:** Specifies how many seconds or milliseconds an animation takes to complete
- ✓ **animation-timing-function:** Specifies the speed curve of the animation.
- ✓ **animation-delay:** Specifies a delay before the animation will start.
- ✓ **animation-iteration-count:** Specifies how many times an animation should be played.
- ✓ **The @keyframes Rule** When you specify CSS styles inside the @keyframes rule, the animation will gradually change from the current style to the new style at certain times.

CSS Styles – Animatable Examples

```
<!DOCTYPE html>
<html> <head> <style>
div {
width: 100px;
height: 100px;
background: red;
position: relative;
animation: move 5s infinite;
}
@keyframes move {
from {left: 0px;}
to {left: 200px;}
}
</style> </head>
```

```
<body>
<h1>The animation Property</h1>
<p>Move an element from 0px to 200px left. The animation lasts for 5 seconds. It then starts over again, and go on forever (infinite).</p>
<div></div>
<p><strong>Note:</strong> The animation property is not supported in Internet Explorer 9 and earlier versions.</p>
</body>
</html>
```

86

CSS Styles – Animatable Examples

Ex1

```
<!DOCTYPE html>
<html> <head> <style>
body {
background-color: red;
animation-name: changeBG;
animation-duration: 14s;
animation-fill-mode: forwards; }
@keyframes changeBG {
from {background-color: red;}
to {background-color: yellow;}
}
</style> </head>
<body>
<p><b>Note:</b> This example does not work in Internet Explorer 9 and earlier versions.</p>
</body>
</html>
```

Ex2

```
<style>
div {
width: 100px;
height: 100px;
background-color: red;
animation-name: example;
animation-duration: 14s;
}
@keyframes example {
0% {background-color: red;}
25% {background-color: yellow;}
50% {background-color: blue;}
100% {background-color: green;}
}
</style>
```

87

CSS Styles – Animatable Examples

```
<!DOCTYPE html>
<html> <head> <style>
div { width: 100px;
height: 100px;
background-color: red;
position: relative;
animation-name: example;
animation-duration: 4s; animation-direction: alternate-reverse; }
@keyframes example { 0% {background-color:red; left:0px; top:0px;}
25%{background-color:yellow; left:200px; top:0px;}
50% {background-color:blue; left:200px; top:200px;}
75% {background-color:green; left:0px; top:200px;}
100% {background-color:red; left:0px; top:0px;} }
</style> </head>
<body> <div></div> </body> </html>
```


CSS Styles – Animatable Examples

```
<!DOCTYPE html>
<html>
<head>
<style>
div { width: 100px;
height: 50px;
background-color: red;
font-weight: bold;
position: relative;
animation: mymove 5s infinite; }
@keyframes mymove {
from {left: 0px;}
to {left: 300px;} }
#div1 {animation-timing-function: linear;}
#div2 {animation-timing-function: ease;}
```

```
#div3 {animation-timing-function: ease-in;}
#div4 {animation-timing-function: ease-out;}
#div5 {animation-timing-function: ease-in-out;}
</style>
</head>
<body>
<div id="div1">linear</div>
<div id="div2">ease</div>
<div id="div3">ease-in</div>
<div id="div4">ease-out</div>
<div id="div5">ease-in-out</div>
</body>
</html>
```